

JACKUT

Rede Social de Relacionamentos

Relatório de Design — Milestone 2

Disciplina: Programação 2
Universidade Federal de Alagoas — UFAL
Autora: Laura Beatriz Lins Ramos Mainero
2026

1. Introdução

Este relatório descreve o design e as decisões arquiteturais do Milestone 2 do sistema Jackut, uma rede social de relacionamentos desenvolvida para a disciplina de Programação 2 da UFAL.

O Milestone 2 expandiu o sistema com cinco novas User Stories (US5 a US9), cobrindo comunidades, mensagens em grupo, novos tipos de relacionamento (fã/ídolo, paquera e inimizade) e remoção de conta em cascata. Além das novas funcionalidades, a arquitetura foi significativamente refinada para corrigir os pontos penalizados no Milestone 1 e aplicar princípios mais rigorosos de Orientação a Objetos.

2. Correções em Relação ao Milestone 1

Três penalidades foram apontadas na correção do Milestone 1. Todas foram corrigidas neste milestone:

Penalidade	Problema no M1	Correção no M2
Exception sem implementação adequada	JackutException estendia RuntimeException (unchecked)	JackutException agora estende Exception (checked), obrigando declaração explícita via throws
Modularidade e Separação de Responsabilidades	Controlador acumulava sessões, persistência e negócio	GerenciadorSessoes e Repositorio extraídos como classes independentes
Uso inconsistente de package	Classes no pacote padrão e em entities/exceptions sem raiz comum	Todos os pacotes sob jackut.*: controllers, services, repositories, entities, exceptions

3. User Stories Implementadas

US	Título	Descrição	Milestone	Status
US1	Gestão de Contas	Criação de conta e autenticação via sessão UUID.	1	✓
US2	Edição de Perfil	Atributos livres de perfil (nome, descrição, cidade natal, etc.).	1	✓
US3	Amizades	Convite bilateral; confirmada quando ambos se adicionam.	1	✓
US4	Recados	Recados privados entre usuários em fila FIFO.	1	✓

US5	Criação de Comunidades	Nome único, descrição e dono; dono é automaticamente o primeiro membro.	2	✓
US6	Participação em Comunidades	Adesão de usuários a comunidades existentes.	2	✓
US7	Mensagens em Comunidades	Mensagem enviada a comunidade é entregue a todos os membros em fila FIFO.	2	✓
US8	Novos Relacionamentos	Fã/ídolo, paquera (notificação automática de match mútuo) e inimizade (bloqueia interações).	2	✓
US9	Remoção de Conta	Remove conta com cascata: dissolve comunidades, limpa relacionamentos e descarta recados pendentes.	2	✓

4. Arquitetura do Sistema

4.1 Estrutura de Pacotes

A reorganização de pacotes foi a principal correção estrutural do Milestone 2. Toda a aplicação está sob o pacote raiz `jackut`, com subpacotes de propósito único:

Pacote	Responsabilidade
<code>jackut</code>	Facade (porta de entrada pública) e Main (executor de testes)
<code>jackut.controllers</code>	Controlador: orquestra o fluxo de negócio entre entidades
<code>jackut.services</code>	GerenciadorSessoes: ciclo de vida das sessões ativas
<code>jackut.repositories</code>	Repositorio: serialização em disco; PersistenceException: erro de I/O
<code>jackut.entities</code>	Usuario e Comunidade: modelos ricos de domínio
<code>jackut.exceptions</code>	JackutException (base checked) + 22 exceções atômicas de negócio

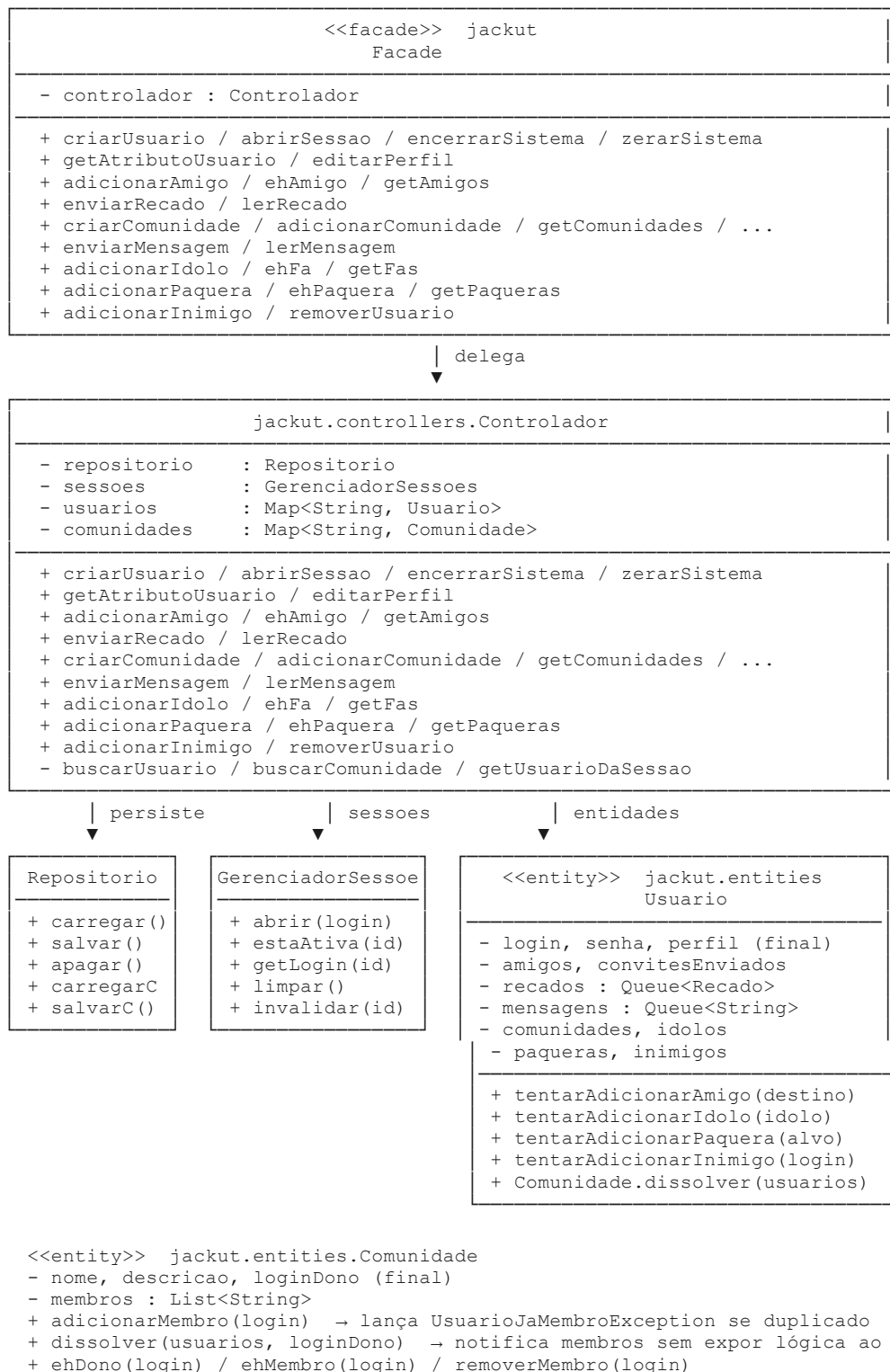
4.2 Camadas e Fluxo de uma Chamada

Toda chamada do EasyAccept percorre o seguinte fluxo:

- **EasyAccept** chama um método da Facade.
- **Facade** delega ao Controlador sem nenhuma lógica própria.
- **Controlador** resolve a sessão via GerenciadorSessoes, busca as entidades e ordena a ação.
- **Entidades** (Usuario, Comunidade) executam a ação, validam seu estado e lançam exceções se necessário.
- **Exceções** propagam livremente até a Facade, que as deixa subir para o EasyAccept capturar a mensagem exata.

5. Diagrama de Classes

O diagrama abaixo apresenta as principais classes, atributos, métodos e relacionamentos do sistema no Milestone 2:



```
<<exceptions>> jackut.exceptions  
JackutException extends Exception (checked) ← 22 exceções atômicas  
PersistenciaException extends RuntimeException (infra, separada do domínio)
```

6. Padrões de Projeto e Decisões de Design

6.1 Facade (mantido e refinado)

A Facade continua sendo o único ponto de contato entre o EasyAccept e o sistema. No Milestone 2, com a mudança para exceções checked, todos os métodos da Facade passaram a declarar explicitamente as exceções que podem lançar via throws. Isso torna o contrato público da Facade completamente transparente em tempo de compilação.

6.2 Exceções Checked (JackutException extends Exception)

A principal correção de design do Milestone 2. No Milestone 1, JackutException estendia RuntimeException, o que significava que erros de negócio podiam ser silenciados por qualquer chamador sem aviso do compilador.

Com a mudança para extends Exception, o compilador passa a exigir que cada método declare via throws as exceções que pode lançar. Isso tem dois benefícios concretos: o contrato de cada método fica visível na assinatura, e é impossível esquecer de tratar ou propagar um erro de negócio sem que o código deixe de compilar.

Exceções de infraestrutura (erros de disco) usam PersistenciaException extends RuntimeException deliberadamente — falha de I/O não é uma regra de negócio violada e não faz sentido ser checked.

6.3 Separação de Responsabilidades (extrações do Milestone 1)

No Milestone 1, o Controlador acumulava três responsabilidades independentes: gerenciar sessões, aplicar regras de negócio e persistir dados. Qualquer mudança em uma dessas áreas forçava a edição do mesmo arquivo.

- **GerenciadorSessoes** — extraído para isolar o ciclo de vida das sessões (abrir, verificar, invalidar). O Controlador não sabe como UUIDs são gerados nem como sessões são armazenadas.
- **Repositorio** — já existia no M1, mas foi refinado para persistir usuários e comunidades em arquivos separados (dados.dat e comunidades.dat), cada um com seus métodos carregar/salvar.
- **Controlador** — ficou sem nenhum import de java.io.* e sem lógica de UUID, focado exclusivamente em orquestração de negócio.

6.4 Rich Domain Model e Tell, Don't Ask (evolução do Milestone 1)

No Milestone 1, o modelo de domínio rico estava presente na validação do construtor de Usuario. No Milestone 2, esse princípio foi estendido aos relacionamentos.

O padrão Tell, Don't Ask define que o chamador não deve interrogar o estado de um objeto para decidir o que fazer com ele — deve apenas ordenar uma ação e deixar o objeto decidir se aceita ou rejeita. No Milestone 1, o Controlador fazia sequências de perguntas antes de agir:

```
// Padrão Ask (Milestone 1) — Controlador interroga o modelo
if (remetente.ehAmigo(loginAmigo))          throw new
UsuarioJaAmigoException();
if (remetente.temConvitePara(loginAmigo)) throw new
ConvitePendenteException();
if (destino.temConvitePara(...)) { remetente.confirmarAmizade(...); }
else { remetente.enviarConvite(...); }
```

No Milestone 2, essa lógica foi encapsulada nos próprios modelos:

```
// Padrão Tell (Milestone 2) — Controlador ordena, Usuario decide
remetente.tentarAdicionarAmigo(destino); // lança a exceção certa
internamente
```

Os quatro métodos que aplicam esse padrão no Usuario são:

- **tentarAdicionarAmigo(destino)** — valida auto-adição, inimizade, amizade existente e convite pendente; decide entre enviarConvite e confirmarAmizade.
- **tentarAdicionarIdolo(idolo)** — valida auto-adição, inimizade e duplicidade.
- **tentarAdicionarPaquera(alvo)** — valida auto-adição, inimizade e duplicidade; detecta match mútuo e envia recado automático a ambos.
- **tentarAdicionarInimigo(login)** — valida auto-adição e duplicidade.

6.5 Comunidade como Modelo Rico

No Milestone 1, Comunidade era essencialmente um repositório de dados. No Milestone 2, ganhou duas responsabilidades próprias:

- **adicionarMembro lança UsuarioJaMembroException** — o chamador não precisa verificar ehMembro() antes; a entidade defende seu próprio estado.
- **dissolver(todosUsuarios, loginDono)** — quando o dono encerra a conta (US9), a Comunidade notifica seus próprios membros para remover a referência. O Controlador não precisa navegar pela lista de membros externamente — basta chamar comunidade.dissolver(...). Isso reduziu o método removerUsuario de 40 para 15 linhas.

6.6 Hierarquia de Exceções Atômicas

O Milestone 2 adicionou 11 novas exceções atômicas para cobrir os casos das US5–US9. A hierarquia completa conta com 23 classes, todas em jackut.exceptions:

Exceção	Situação
LoginInvalidoException	Login nulo ou vazio ao criar usuário
SenhaInvalidaException	Senha nula ou vazia ao criar usuário
ContaJaExisteException	Login já cadastrado
LoginOuSenhaInvalidosException	Credenciais incorretas ao abrir sessão
UsuarioNaoCadastradoException	Login ou sessão não encontrados
AtributoNaoPreenchidoException	Atributo de perfil não definido
AutoAmizadeException	Usuário tentando adicionar a si mesmo como amigo
UsuarioJaAmigoException	Amizade já confirmada
ConvitePendenteException	Convite já enviado, aguardando aceite
AutoRecadoException	Envio de recado para si mesmo
SemRecadosException	Fila de recados vazia
ComunidadeJaExisteException	Nome de comunidade já existe
ComunidadeNaoExisteException	Comunidade não encontrada
UsuarioJaMembroException	Usuário já é membro da comunidade
SemMensagensException	Fila de mensagens de comunidade vazia
AutoldoloException	Usuário tentando adicionar a si mesmo como ídolo
UsuarioJaldoloException	Ídolo já na lista
AutoPaqueraException	Usuário tentando adicionar a si mesmo como paquera
UsuarioJaPaqueraException	Paquera já na lista
AutolnimigoException	Usuário tentando declarar a si mesmo como inimigo
UsuarioJalnimigoException	Inimigo já na lista
InimigoException	Interação bloqueada porque o alvo declarou o solicitante como inimigo (mensagem inclui o nome)

7. Decisões Técnicas Relevantes

7.1 Recado armazena o remetente (classe interna Recado)

A fila de recados do Usuario armazena objetos da classe interna Recado (que guarda remetente + texto) em vez de Strings simples. Essa decisão foi motivada pela US9: ao remover um usuário, todos os recados não lidos que ele enviou devem ser descartados da fila dos destinatários. Sem o remetente gravado junto ao recado, essa limpeza seria impossível sem varrer todas as filas sem saber o que remover.

7.2 Persistência em dois arquivos separados

Usuários e comunidades são serializados em arquivos distintos (dados.dat e comunidades.dat). A alternativa seria um único objeto contendo os dois mapas, como no Milestone 1. A separação foi escolhida porque os dois mapas têm ciclos de vida independentes: zerarSistema precisa apagar os dois, mas outras operações afetam apenas um deles. Dois arquivos tornam cada operação mais explícita e reduzem o risco de corrupção cruzada.

7.3 Inimizade bloqueia o alvo, não o declarante

Quando A declara B como inimigo, é B quem fica bloqueado de interagir com A (adicionar como amigo, ídolo, paquera ou enviar recado) — não o contrário. A verificação de inimizade é sempre feita no destino da ação: se o destino tem o solicitante como inimigo, a operação é bloqueada. A mensagem de erro inclui o nome do destino para indicar quem bloqueou a interação.

7.4 Execução dos testes em subprocessos

O EasyAccept chama System.exit() ao encerrar cada script, o que encerraria a JVM principal. A classe Main contorna isso lançando cada teste como subprocesso independente via ProcessBuilder. Os arquivos de persistência em disco garantem que os pares _1/_2 de cada US compartilhem estado entre subprocessos.

8. Qualidade e Testes

O projeto foi validado com 100% de aprovação em todos os testes de aceitação do EasyAccept:

Script	Descrição	Testes	Resultado	Tipo
us1_1	Criação de conta e sessão	17	✓ OK	Funcional
us1_2	Persistência de contas	7	✓ OK	Persistência
us2_1	Edição de perfil	36	✓ OK	Funcional
us2_2	Persistência de perfil	13	✓ OK	Persistência
us3_1	Sistema de amizades	46	✓ OK	Funcional
us3_2	Persistência de amizades	10	✓ OK	Persistência
us4_1	Envio e leitura de recados	42	✓ OK	Funcional
us4_2	Persistência de recados	13	✓ OK	Persistência
us5_1	Criação de comunidades	31	✓ OK	Funcional
us5_2	Persistência de comunidades	13	✓ OK	Persistência
us6_1	Participação em comunidades	28	✓ OK	Funcional
us6_2	Persistência de participação	7	✓ OK	Persistência

us7_1	Mensagens em comunidades	68	✓ OK	Funcional
us7_2	Persistência de mensagens	16	✓ OK	Persistência
us8_1	Novos relacionamentos	95	✓ OK	Funcional
us8_2	Persistência de relacionamentos	7	✓ OK	Persistência
us9_1	Remoção de conta	17	✓ OK	Funcional
us9_2	Persistência pós-remoção	7	✓ OK	Persistência
Total		476	476 ✓	

9. Conclusão

O Milestone 2 entregou as cinco novas User Stories e corrigiu todas as penalidades do Milestone 1. A arquitetura evoluiu em três frentes principais: a hierarquia de exceções tornou-se checked, garantindo contratos visíveis em tempo de compilação; a estrutura de pacotes ficou consistente sob jackut.*; e os modelos de domínio ganharam responsabilidade real sobre seu próprio estado com o padrão Tell, Don't Ask.

O resultado é um sistema que compila, passa 476/476 testes e apresenta cada classe com um único motivo claro de mudança — o que facilita a expansão para milestones futuros sem que novas funcionalidades precisem tocar em código já estabilizado.